

VS Code Home Lab Setup

Remote-SSH Configuration - Viewer Guide

Author: Dalla Samuel (CyberJKD)

Date: September 18, 2026

Course: Cloud System Admin Accelerator - How To Automate The VM Deployment - VSCode Guide

Video: [VS Code Home Lab Setup](#)

Roadmap: Phase 06 - Cloud Tech Techniques - Cloud System Admin Accelerator

GitHub: tinyurl.com/vscode-remote-ssh-setup

How to Use This Guide

This document mirrors the video walkthrough step for step.

Each ■ marker is a checkpoint - a point in the video where a real decision, command, or fix happened. Pause the video at each one, run the same command on your own machine, and compare your output (and the screenshot below each checkpoint) against the “Expect” line before moving on.

This session was not a clean, scripted run either.

Two real failures happen along the way - a Git clone that fails mid-transfer, and an SSH connection that gets refused because the remote service wasn't running.

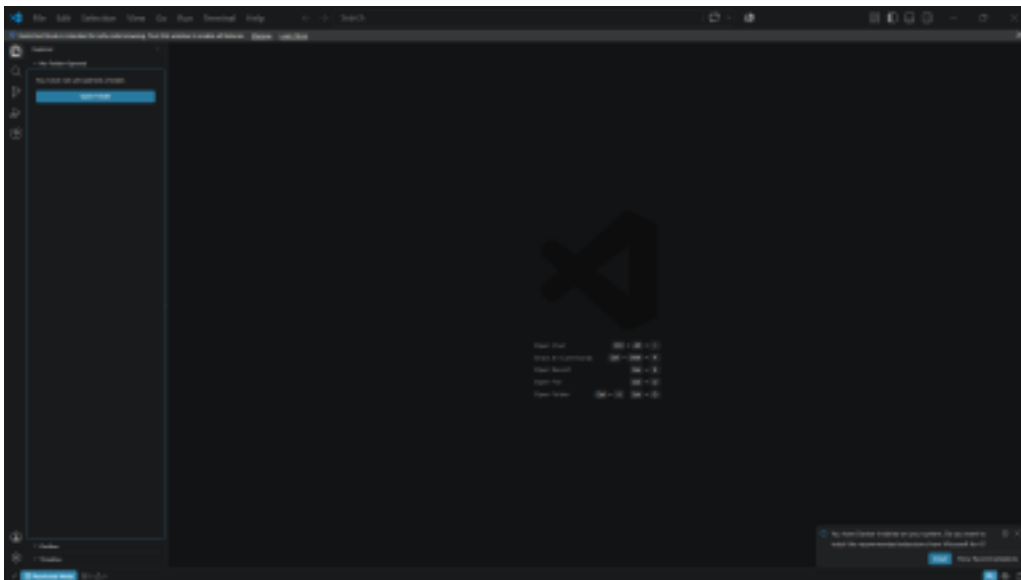
They're left in deliberately, with real screenshots of each, because diagnosing them is most of what this session actually teaches: how to tell a network problem from an auth problem from a service-not-running problem.

Part 1 - Prerequisites

■ 00 Confirm VS Code is installed

VS Code was already installed on this machine. First launch of a fresh workspace shows the Welcome tab, a Restricted Mode banner (no folder trusted yet), and — since Docker was already present on this system — a prompt offering Microsoft's recommended Docker extensions.

Expect: Restricted Mode banner visible; Docker extension prompt in the bottom-right corner.

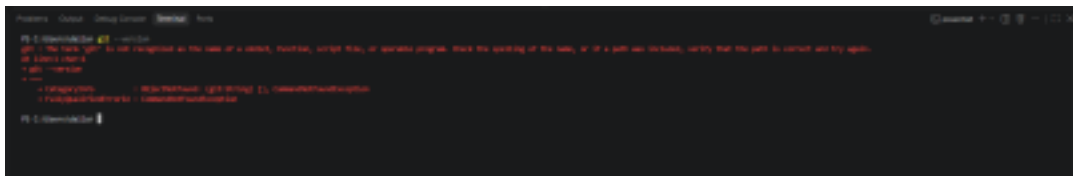


Fresh VS Code launch, Restricted Mode, Docker prompt

■ 01 Verify Git

Open the integrated terminal (Ctrl+`) and run `git --version`.

Expect: Either a version number, or an error that git isn't recognized if it isn't installed yet.



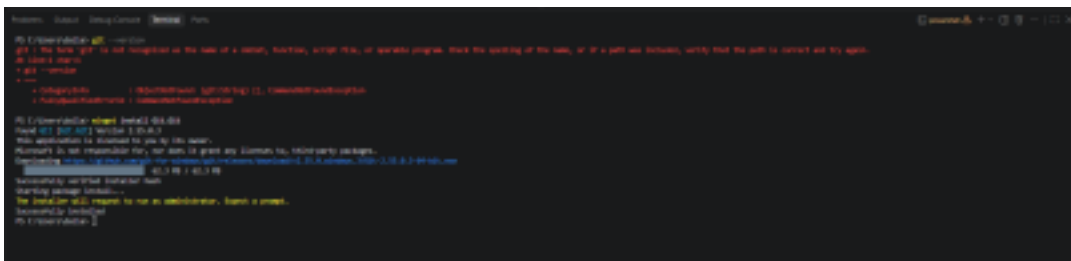
```
PS C:\Users\user> git --version
git: 'git' is not recognized as the name of a built-in function, script file, or executable program. Check the spelling of the name, or if a path was included, verify that the path is correct and try again.
PS C:\Users\user>
```

git

--version not recognized

If not installed, run `winget install Git.Git`.

Expect: Download and install complete, version confirmed (this session: 2.55.0.3).



```
PS C:\Users\user> winget install Git.Git
Found 1: [Git.Git] version 2.55.0.3
This application is licensed by its file name.
Microsoft is not responsible for any data or content that is posted to any package.
Downloading https://aka.ms/winget-pkgs/Git.Git/2.55.0.3
Successfully installed Git.Git
Starting package installation...
The package is installed successfully.
PS C:\Users\user>
```

winget Git install output

Part 2 - Core Extensions

■ 02 Install extensions via terminal

Rather than clicking through the Extensions panel one at a time, install everything needed in one batch from the terminal:

```
code --install-extension ms-vscode-remote.remote-ssh
```

```
code --install-extension ms-vscode.powershell
```

```
code --install-extension redhat.vscode-yaml
```

```
code --install-extension ms-python.python
```

```
code --install-extension eamodio.gitlens
```

```
code --install-extension hashicorp.terraform
```

Expect: Each extension reports "successfully installed"; the PowerShell extension banner appears in a fresh terminal confirming it's active.



```
code --install-extension ms-vscode-remote.remote-ssh --code --install-extension ms-vscode.powershell --code --install-extension redhat.vscode-yaml --code --install-extension ms-python.python --code --install-extension eamodio.gitlens --code --install-extension hashicorp.terraform
```

The terminal output shows the following sequence of events:

- VS Code starts with a warning about the Microsoft account.
- The command to install extensions is executed.
- Each extension is installed successfully, with the following output for each:
 - Installing extension 'ms-vscode-remote.remote-ssh' v0.0.10 successfully installed.
 - Installing extension 'ms-vscode.powershell' v1.14.0 successfully installed.
 - Installing extension 'redhat.vscode-yaml' v0.18.0 successfully installed.
 - Installing extension 'ms-python.python' v2020.10.0 successfully installed.
 - Installing extension 'eamodio.gitlens' v7.18.0 successfully installed.
 - Installing extension 'hashicorp.terraform' v0.14.0 successfully installed.

All extensions installed successfully



```
PowerShell Extension v1.14.0
Copyright (c) Microsoft Corporation.
All rights reserved.
Type 'help' for get help.
PS C:\Users\jdoe>
```

The screenshot shows a fresh terminal window with the PowerShell extension banner displayed at the top, indicating it is active.

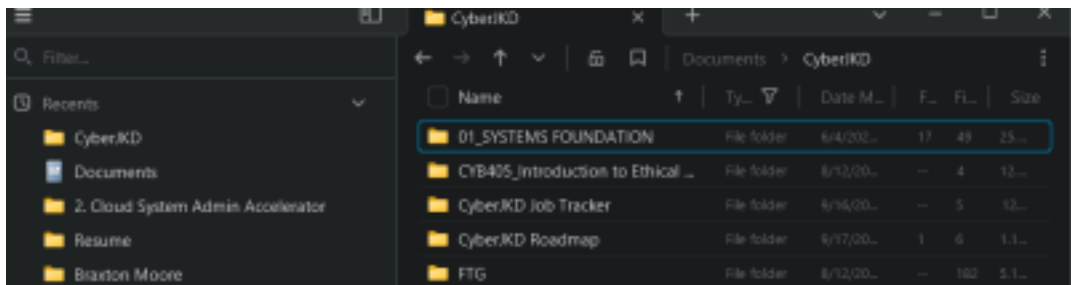
PowerShell extension active in terminal

Part 3 - Clone the Repo

■ 03 Confirm the repo isn't local yet

Check the working folder (here, Documents\CyberJKD) before cloning — this session already had several other CyberJKD project folders present, but not CyberJKD-Labs itself.

Expect: The target repo folder is absent from the directory listing.



CyberJKD folder contents - no Labs repo present

■ 04 Clone and expect a possible transfer failure

e

On a repo of this size (1249 objects), it's common to hit an HTTP buffer error mid-transfer — a live network/protocol limit, not a mistake in the command.

```
cd "$HOME\Documents\CyberJKD"
```

```
git clone https://github.com/DallaSamuel/CyberJKD-Labs.git
```

Expect: "error: RPC failed; curl 18 transfer closed with outstanding read data remaining" followed by "fatal: early EOF".

```
PS C:\Users\della\Documents\Cyber30> git clone https://github.com/DallaSamuel/Cyber30-Labs.git
Cloning into 'Cyber30-Labs'...
remote: Enumerating objects: 1249, done.
remote: Counting objects: 100% (1249/1249), done.
remote: Compressing objects: 100% (1249/1249), done.
remote: Total 1249 (delta 119), reused 48 (delta 48), pack-reused 949 (from 2)
Receiving objects: 100% (1249/1249), 27.51 MiB | 526.80 KiB/s, done.
Resolving deltas: 100% (427/427), done.
PS C:\Users\della\Documents\Cyber30>
```

Failed clone - RPC error, early EOF

■ 05 Raise the buffer size and retry

Run these as two separate commands - pasting them combined on one line causes PowerShell to misparse the second as an argument to the first, producing "error: no action specified".

```
git config --global http.postBuffer 524288000
```

```
git clone https://github.com/DallaSamuel/Cyber30-Labs.git
```

Expect: "Receiving objects: 100% (1249/1249), done." with no errors.

```
PS C:\Users\della\Documents\Cyber30> git config --global http.postBuffer 524288000
error: no action specified
PS C:\Users\della\Documents\Cyber30> git config --global http.postBuffer 524288000
PS C:\Users\della\Documents\Cyber30> git clone https://github.com/DallaSamuel/Cyber30-Labs.git
Cloning into 'Cyber30-Labs'...
remote: Enumerating objects: 1249, done.
remote: Counting objects: 100% (1249/1249), done.
remote: Compressing objects: 100% (1249/1249), done.
remote: Total 1249 (delta 119), reused 48 (delta 48), pack-reused 949 (from 2)
Receiving objects: 100% (1249/1249), 27.51 MiB | 526.80 KiB/s, done.
Resolving deltas: 100% (427/427), done.
PS C:\Users\della\Documents\Cyber30>
```

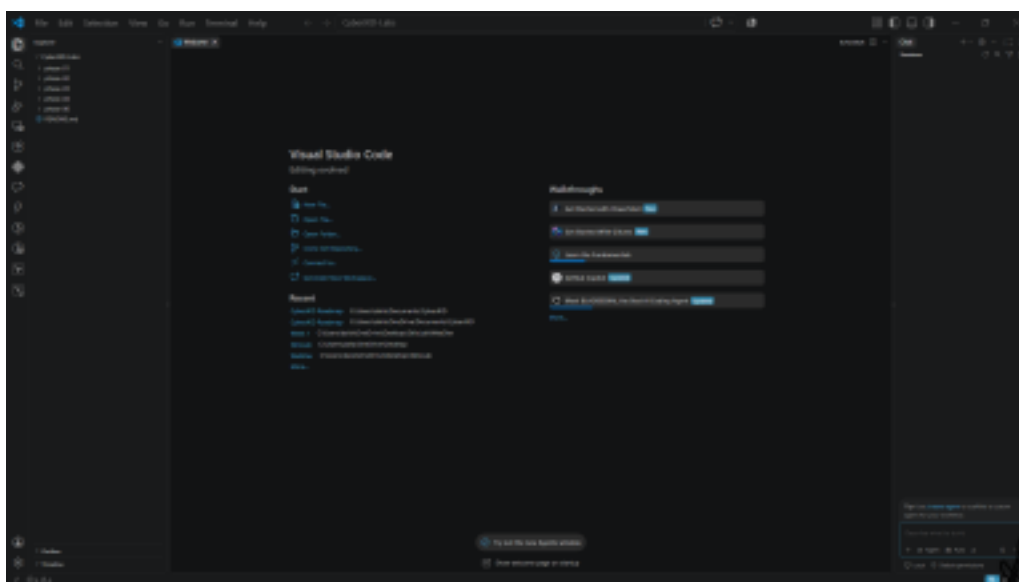
Successful clone after buffer fix

Part 4 - Open and Trust

■ 06 Open the cloned folder

File → Open Folder → select CyberJKD-Labs. Cloning a repo directly (rather than downloading a zip) auto-trusts the workspace, so the Restricted Mode banner does not reappear.

Expect: Sidebar shows phase-01 through phase-06 and README.md; no Restricted Mode banner.



CyberJKD-Labs open in the sidebar

Part 5 - Remote-SSH to the Kali VM

■ 07 Configure the SSH host entries

Ctrl+Shift+P → **Remote-SSH: Open SSH Configuration File** → select the personal config (C:\Users\<you>\.ssh\config, not the system-wide ProgramData one). Add a Host block per VM:

Host kali

 HostName 192.168.56.102

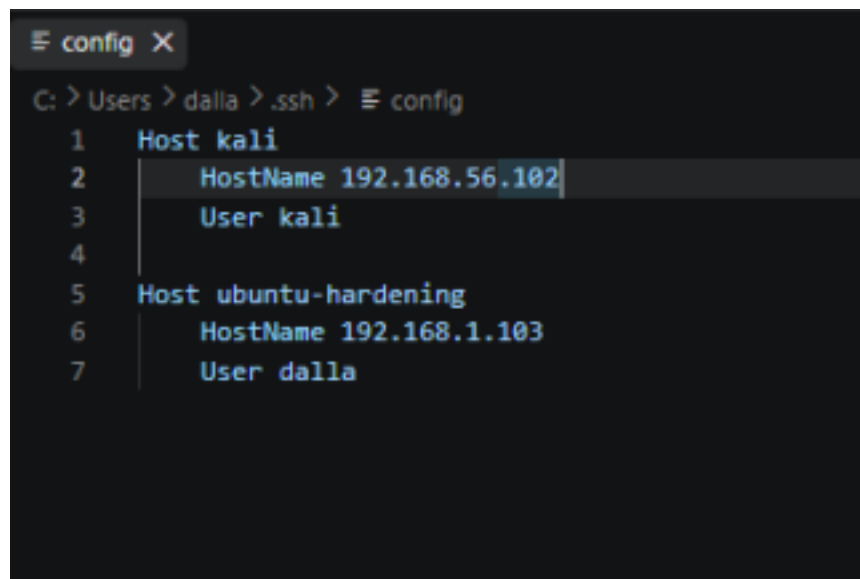
 User kali

Host ubuntu-hardening

 HostName 192.168.1.103

 User dalla

Expect: Both host blocks saved with correct indentation, no syntax errors.

A screenshot of a terminal window with a dark background. The title bar at the top shows a hamburger menu icon, the text 'config', and a close button 'X'. The terminal shows the command prompt 'C: > Users > dalla > .ssh > ≡ config' followed by the contents of the SSH config file. The file contains two host blocks. The first block is for 'Host kali' with 'HostName 192.168.56.102' and 'User kali'. The second block is for 'Host ubuntu-hardening' with 'HostName 192.168.1.103' and 'User dalla'. Line numbers 1 through 7 are visible on the left side of the terminal window.

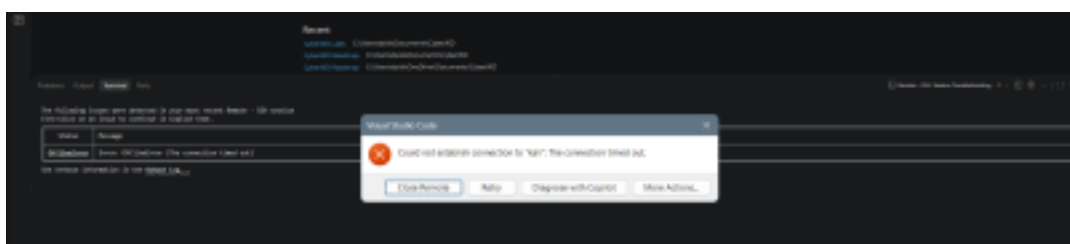
```
≡ config X
C: > Users > dalla > .ssh > ≡ config
1  Host kali
2      HostName 192.168.56.102
3      User kali
4
5  Host ubuntu-hardening
6      HostName 192.168.1.103
7      User dalla
```

SSH config with both host entries

■ 08 First connection attempt: wrong network

Ctrl+Shift+P → **Remote-SSH: Connect to Host** → kali. The Kali VM in this session was actually running on the CYB 405 host-only lab network (192.168.56.x), not the home-lab range (192.168.1.x) the config was first pointed at.

Expect: "Could not establish connection to 'kali': The connection timed out."



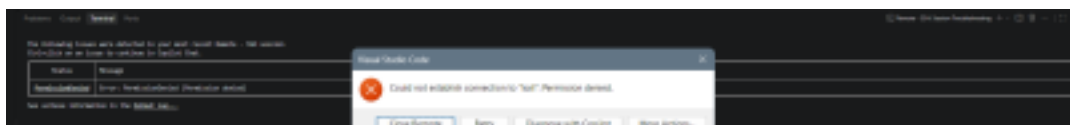
Connection timed out - wrong IP

■ 09 Diagnose with a raw SSH attempt

After correcting the HostName to the right subnet, the Remote-SSH extension can still return a misleading result - "Permission denied" with no password prompt, which looks like an auth problem but often isn't. Bypass the extension and test directly in the terminal:

```
ssh kali@192.168.56.102
```

Expect: The real underlying error - in this session, a "Connection refused" on port 22, meaning the SSH server wasn't even running.



Permission denied via Remote-SSH extension

A terminal window with a dark background and light text. The text shows a raw SSH connection attempt from a host to a Kali VM. The output indicates that the connection was refused on port 22. The terminal has tabs at the top labeled 'Hosts', 'Output', 'Running Commands', and 'SSH'.

```
PS C:\Users\jacob\Documents> ssh -o StrictHostKeyChecking=no kali@192.168.56.102
ssh: connect to host 192.168.56.102 port 22: Connection refused
PS C:\Users\jacob\Documents>
```

Raw ssh — connection refused, port 22

■ 10 Start SSH on the Kali VM itself

Switch to the Kali VM's own window (not VS Code) and run, directly on Kali. Kali does not run SSH by default - start brings it up for the current session, enable makes it persist across reboots.

```
sudo systemctl start ssh
```

```
sudo systemctl enable ssh
```

```
sudo systemctl status ssh
```

Expect: "active (running)" in the status output.

■ 11 Confirm with a raw SSH login

Retry `ssh kali@192.168.56.102` from the host terminal. Accept the host key fingerprint on first connection, then enter the password.

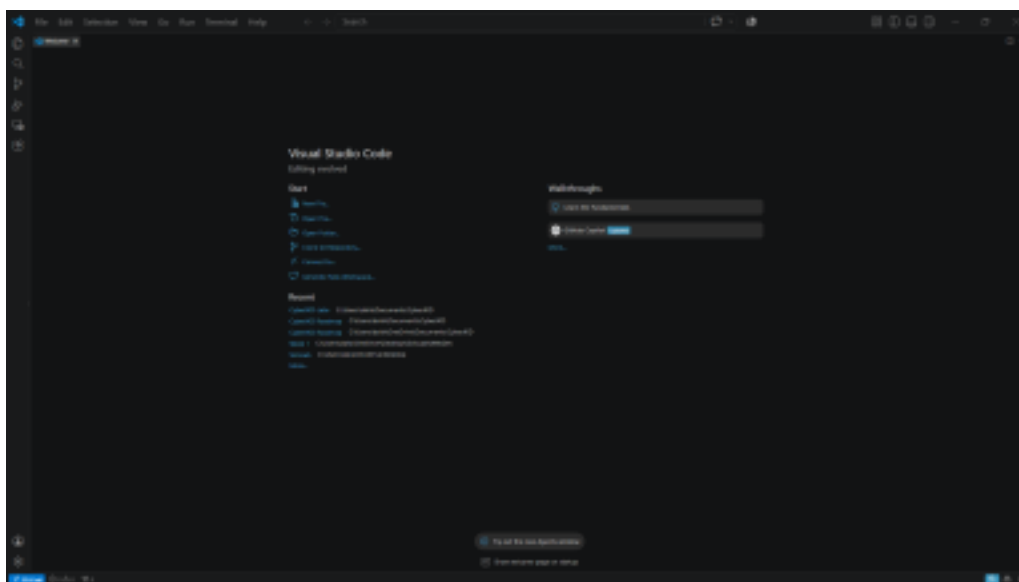
Expect: Host key accepted, password prompt appears, login succeeds and drops into the Kali shell (`kali@kali`).



■ 12 Connect via the Remote-SSH extension

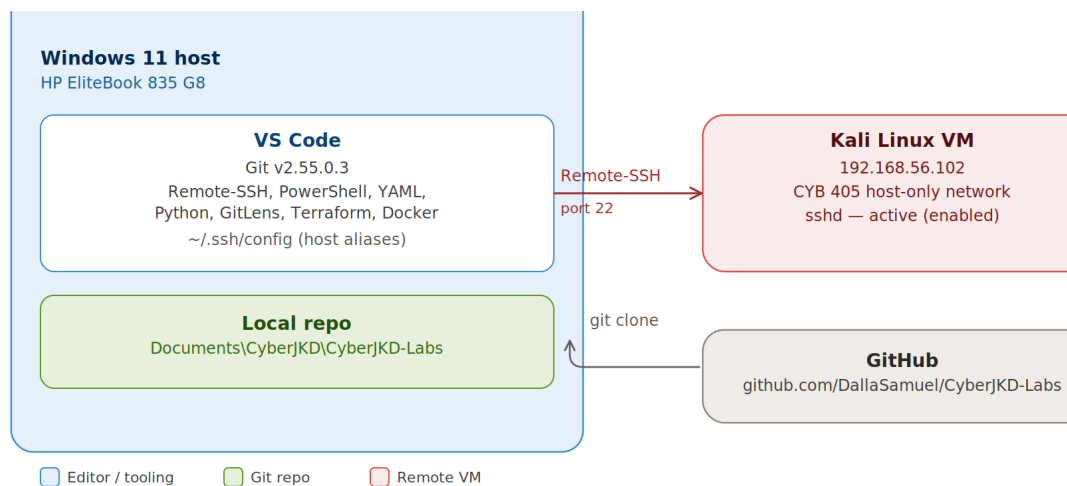
With SSH confirmed working directly, retry through VS Code: Ctrl+Shift+P → **Remote-SSH: Connect to Host** → kali.

Expect: A new VS Code window opens connected to the VM; the status bar (bottom-left) shows green "SSH: kali".



Remote-SSH connected - green "SSH: kali" status bar

Architecture



Host, editor, local repo, and remote VM relationship

Closing Note

Both failures documented here were real - nothing staged. If your own run hits a different error than the ones listed (a different IP scheme, a different missing service), that's expected too; every home lab's network layout is slightly different.

The method that matters more than either specific fix is the one used throughout: when a GUI error looks like one thing (a permission problem) but doesn't quite add up (no password prompt), drop to the raw command-line tool and get the unfiltered error before assuming the cause.

Full roadmap:

dallasamuel.github.io/CyberJKD-Roadmap

All labs:

github.com/DallaSamuel/CyberJKD-Labs

CyberJKD - Becoming dangerous through fundamentals.